
SPECTRUM BACK OFFICE FUNCTIONAL TESTS

How the AI Agents Work

A practical guide to the instruction-driven VS Code agents created for running, diagnosing, and maintaining the Spectrum Back Office functional test suite.

Prepared: 23 September 2026

Repository: spectrum-backoffice-functional-tests

Agent profiles: 10 user-invokable Markdown profiles in `.github/agents`

1. What Was Created

The project contains ten specialized agent profiles under `.github/agents`. They are instructions for an AI coding agent in VS Code, not a second test framework and not autonomous services running in the application.

Agent	Primary responsibility
All Features Runner	Runs and diagnoses every feature under <code>features/functional/*.feature</code> .
Appointments Feature Runner	Owns the appointments feature and its UI flow.
Bulk Feature Runner	Owns bulk import list and bulk details validation.
Disciplines Feature Runner	Owns discipline reference-table behavior.
Organization Types Feature Runner	Owns organization type reference-table behavior.
Organizations Feature Runner	Owns organization CRUD and related workflows.
Reports Feature Runner	Owns embedded reports and iframe behavior.
Requests Feature Runner	Owns requests, cancelled requests, filters, pagination, and CRUD behavior.
Settings Feature Runner	Owns application settings tabs and validation.
Venues Feature Runner	Owns venue search, filtering, validation, and creation.

Important boundary: the agents coordinate existing tests. The business behavior remains in Gherkin feature files, Cucumber steps, Playwright page objects, hooks, and utilities.

2. How an Agent Profile Is Built

Each `.agent.md` file has two parts: metadata that makes the profile discoverable in VS Code, followed by the operating instructions that shape the agent's behavior.

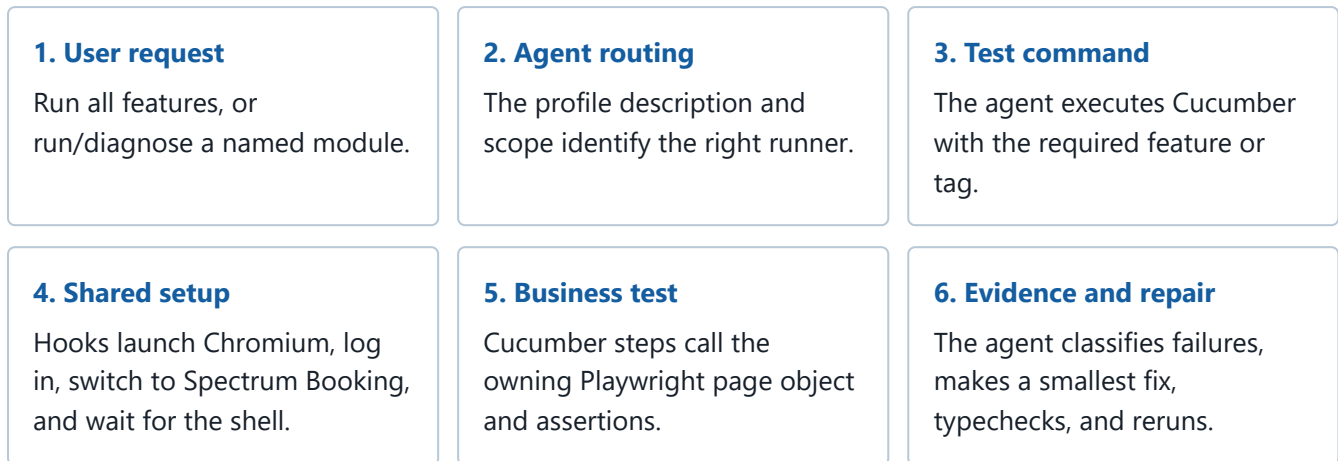
Metadata

- `name`: the label shown to the user.
- `description`: the routing signal. It states which request wording, feature, and failure types should activate the agent.
- `tools`: `[read, search, execute, edit]`: the intended capabilities.
- `user-invocable`: `true`: the profile can be selected or invoked directly by the user.

Instructions

The body gives the agent an explicit mission, scope, safety rules, commands, diagnosis workflow, and output format. This is what turns a general coding agent into a repeatable feature runner.

3. Runtime Flow



The execution path is therefore:

```
agent profile -> npm / cucumber-js -> Gherkin feature -> Cucumber step -> Page Object -> Playwright browser -> live Spectrum Back Office
```

4. The Shared Test Lifecycle

1. `BeforeAll` confirms the required environment is present without printing secrets.
2. The tagged `@spectrum` hook launches Chromium and creates a scenario-scoped browser context and page.
3. `LoginPage` submits the configured username and password.
4. `EntitlementsPage` waits for readiness and switches to `Spectrum Booking`.
5. `SpectrumBackOfficePage` waits for the application shell.
6. Feature steps perform business actions through module-specific page objects.
7. On failure, the hook attaches a full-page screenshot, then closes the context and browser.

Secrets: credentials come from local environment configuration. Agents are explicitly instructed never to read, print, summarize, or place passwords, cookies, tokens, or authorization headers in commands, source, screenshots, logs, or reports.

5. Diagnosis and Fix Strategy

The agents use a controlled loop rather than changing selectors immediately:

1. Run the complete requested feature first and capture the exit code and Cucumber summary.
2. Inventory every failure. The all-features runner must not stop at the first failure.
3. Isolate a failure by feature, then by exact scenario name.
4. Identify the failed step, owning file and method, locator or assertion, and phase: setup, test body, teardown, or runner.
5. Classify the cause: configuration, environment, authentication, navigation, selector, timing, data, assertion, teardown, or infrastructure.

6. Apply the smallest evidence-based change in the owning hook, page object, step, utility, or setup.
7. Run `npm run typecheck`, then rerun the complete feature or suite.
8. Stop with a precise live-environment blocker when the evidence points outside the test code.

Agents are specifically told not to add arbitrary sleeps, weaken assertions, skip scenarios, broaden selectors without evidence, or treat stale report files as proof of a current run.

6. Commands Used by the Agents

Purpose	Command
Complete suite	<code>\$env:HEADED = 'false'; npm run test:headless</code>
Feature isolation	<code>cucumber-js.cmd features\\functional\\<feature>.feature --format progress</code>
Exact scenario isolation	<code>cucumber-js.cmd ... --name "<exact scenario name>" --format progress</code>
Type safety after a fix	<code>npm run typecheck</code>

`scripts/run-tests.cjs` checks for `SBPROJECT_URL`, `SBPROJECT_USERNAME`, and `SBPROJECT_PASSWORD`, forces headless mode, and delegates to Cucumber. `cucumber.js` loads TypeScript support, hooks, and steps, runs serially, and writes the standard JSON report.

7. Why There Are Specialized Agents

Specialization reduces the amount of context each run needs and limits edits to the owning module. A requests failure should lead to `RequestsPage` or its steps, while a venues failure should remain in the venues slice. The all-features profile provides broad regression coverage and is the final verification authority when a shared change is made.

The result is a simple division of responsibility:

Layer	Responsibility
Agent profile	Decides how to run, diagnose, edit, verify, and report.
Feature file	Defines business intent in Gherkin.
Step definition	Maps Gherkin language to test actions.
Page object	Owns locators and UI interactions.
Hook and world	Owns browser lifecycle, authentication, and scenario state.
Playwright and Cucumber	Execute the browser flow and report outcomes.

8. How to Use the Agents

1. Open the repository in VS Code with dependencies installed and the local environment configured.
2. Select the relevant agent profile from the agent picker, or make a request matching its description.

3. Ask for a run, diagnosis, or authorized fix, such as “run the Requests feature and diagnose failures.”
4. Review the command, failure classification, changed file, typecheck result, and rerun result in the agent response.

Expected outcome: a useful result is more than “passed” or “failed.” It identifies scenarios, phases, evidence, fixes, verification, and any remaining environment blocker.

9. Maintenance Rules

- Keep descriptions specific enough for reliable routing.
- Keep module agents narrow and preserve feature intent.
- Update required commands when scripts or feature paths change.
- Keep safety rules aligned with the project secrets policy.
- Preserve the complete-suite verification requirement for shared changes.
- Update the output template when new evidence or failure phases become important.

Generated from the current Spectrum Back Office functional test project. This guide documents the repository's instruction-driven agent profiles and their integration with the existing test architecture.